SIMATS ENGINEERING

Engineer to Excel

SIMATS

Saveetha Institute of Medical And Technical Sciences

(Declared as Deemed to be University under Section 3 of UGC Act 1956)

Department of Computer Science and Engineering

Course Code & Name	CSA06 – Design and Analysis of Algorithms
Assessment	CO1 – Assessment Tool 1: IT Application-Based Problem Solving – Scalability Audit of an E-Commerce Platform
Course Outcome Assessed	Determine algorithm efficiency using asymptotic notations and mathematical techniques including Master Theorem and substitution method.
Duration	3 Days
Mode	Individual
Student Reg. No.	192511308
Student Name	Divya Dharshini V S

Instructions: Answer all parts. Show the basic operation, every step of each derivation and the final asymptotic result, and interpret what each result means for the platform. Assume the platform's servers execute 10^8 basic operations per second.

IT APPLICATION-BASED PROBLEM

Scenario – Scalability Audit of "ShopSphere", an Online Shopping Platform: ShopSphere is a growing e-commerce platform whose product catalogue currently holds n = 100,000 products and is expected to reach n = 1,000,000 within a year. Before the expansion, the CTO has asked you, as the platform's algorithm engineer, to audit five back-end modules, predict how each will behave as n grows, and recommend which modules must be redesigned. The modules are described below.

Module M1 – Duplicate-Listing Detector: Sellers sometimes upload the same product twice. Every night, the following routine compares every product with every later product.

```
FOR i ← 0 TO n − 2 DO
  FOR j ← i + 1 TO n − 1 DO
    IF Product[i].title = Product[j].title AND Product[i].seller = Product[j].seller THEN
      MarkDuplicate(j)
    END IF
```

```
  END FOR
END FOR
```

Module M2 – API Rate Limiter: The public API audits request windows of doubling size using the routine below. A team member claims that it is Θ(n log n) because the outer loop runs about log n times and the inner loop runs up to n times.

```
i ← 1
WHILE i < n DO
  FOR j ← 0 TO i − 1 DO
    Audit(Request[j])          // basic operation
  END FOR
  i ← i × 2
END WHILE
```

Module M3 – Product Search: Two versions exist. Version S1 scans the unsorted catalogue sequentially until the product is found. Version S2 searches a sorted product index by comparing the key with the middle entry and discarding half the index after each comparison; its comparison count satisfies $T(n) = T(n/2) + 1$, $T(1) = 1$.

Module M4 – Recommendation Engine: The engine splits the customer–product data into four parts of size n/2 (because of replicated partitions) and merges the partial results in linear time: $T(n) = 4T(n/2) + n$. A proposed redesign removes the replication, giving $T(n) = 2T(n/2) + n$.

Module M5 – Sales Analytics Engine: The analytics engine splits the sales data into halves recursively and sorts the records by revenue at every level, giving $T(n) = 2T(n/2) + n \log n$. Separately, profiling shows that the nightly report generator performs $f(n) = 3n^2 + 10n \log_2 n + 50$ operations.

(a) Understand the Problem: For each of the five modules, identify the input size and the basic operation that dominates the running time, explain where it occurs, and state how you expect its execution count to grow as n increases. Justify each choice briefly.

(b) Analyse the Iterative Modules: For M1, derive the exact number of comparisons as a function of n and state the time and space complexity. For M2, derive the exact number of Audit operations, decide whether the team member's Θ(n log n) claim is correct and give the tight bound. Verify your M2 result for n = 1,024.

(c) Solve a Recurrence by Substitution: Solve the recurrence for search version S2 using backward substitution, showing every intermediate step, and derive the exact closed form. State the time complexity and the space complexity of both an iterative and a recursive implementation.

(d) Apply the Master Theorem: For M4, apply the Master Theorem to both the current design and the proposed redesign, clearly identifying a, b, f(n), n^(log_b a) and the case used, and

explain what happens to the running time when the data doubles. For M5, determine whether any of the three standard cases applies to $T(n) = 2T(n/2) + n \log n$; if not, derive the complexity using the extended Master Theorem or a recursion tree.

(e) Analyse Best, Average and Worst Cases: For search versions S1 and S2, identify the input condition that produces the best, average and worst case and give the number of comparisons in each. For S1, derive the average number of comparisons when the probability that the searched product exists is p, and evaluate it for n = 1,000,000 and p = 0.5.

(f) Prove an Asymptotic Bound: Using the formal definitions, prove that the report generator's cost $f(n) = 3n^2 + 10n \log_2 n + 50$ is $\Theta(n^2)$ by finding constants c_1, c_2 and n_0, and show that f(n) is not O(n).

(g) Scalability Decision: Estimate the number of basic operations and the running time of M1, M2, S2, M4 (both designs) and M5 for n = 1,000,000. Identify the modules that will fail to meet a nightly processing window of 10 minutes, and recommend, with justification, which modules must be redesigned and how.

Rubrics – Application-Based Question

Criteria	Weightage (%)	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Problem Context	20	Clearly understands the problem and accurately identifies all requirements	Good understanding with minor gaps	Basic understanding; some requirements unclear	Poor understanding or misinterpretation of the problem
Application of Concepts	30	Accurately applies appropriate concepts to solve complex problems	Applies relevant concepts correctly with minor errors	Applies basic concepts with limited accuracy	Fails to apply appropriate concepts
Problem-Solving Approach	20	Logical, well-structured, and efficient approach	Mostly logical approach with minor gaps	Basic approach with some inconsistencies	Illogical or unclear approach
Accuracy of Solution	20	Completely correct solution with proper justification	Mostly correct with minor errors	Partially correct solution	Incorrect or incomplete solution
Clarity	10	Clear, well-organized, and properly explained solution	Understandable with minor clarity issues	Limited clarity and explanation	Poorly presented and difficult to understand

Mark Evaluation:

Criteria	Weightage (%)	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Problem Context	20				
Application of Concepts	30				
Problem-Solving Approach	20				
Accuracy of Solution	20				
Clarity	10				
Total Marks					

Code of Conduct

I Divya Dharshini V S (192511308) (Name / Reg. No.) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____ Date: 11/10/2026

IT APPLICATION-BASED PROBLEM SOLVING

SCALABILITY AUDIT OF SHOPSPHERE E-COMMERCE PLATFORM

(a) UNDERSTAND THE PROBLEM

1. Introduction

ShopSphere is an online shopping platform whose product catalogue currently contains 100,000 products and is expected to grow to 1,000,000 products. This tenfold increase may significantly affect the performance of its back-end modules.

Asymptotic notations help us describe this growth independently of a particular computer. The input size is represented by n, the number of products or records processed. The basic operation is the main operation performed repeatedly by an algorithm, such as a comparison, an audit, or a merge operation.

2. Identification of Basic Operations

Module	Input size	Dominant basic operation	Expected growth
M1: Duplicate-Listing Detector	Number of products, n	Comparing two products	Quadratic, $\Theta(n^2)$
M2: API Rate Limiter	Number of requests, n	Executing Audit(Request[j])	Linear, $\Theta(n)$
M3: Product Search S1	Number of products, n	Comparing the search key with a product	Linear, $O(n)$worst case
M3: Product Search S2	Number of sorted index entries, n	Comparing the key with the middle entry	Logarithmic, $\Theta(\log n)$
M4: Current Recommendation Engine	Number of data items, n	Processing and merging recursive subproblems	$\Theta(n^2)$
M4: Redesigned Recommendation Engine	Number of data items, n	Processing and merging recursive subproblems	$\Theta(n \log n)$
M5: Sales Analytics Engine	Number of sales records, n	Sorting and merging records	$\Theta(n\log^2 n)$

3. Explanation of Each Module

M1 – Duplicate-Listing Detector

The algorithm compares every product with every product appearing after it. Its basic operation is the comparison of product titles and seller identities. The outer loop selects a product, while the inner loop compares it with all later products. The number of comparisons grows quadratically because the number of pairs increases rapidly as the catalogue grows.

For example, checking 1,000,000 products requires hundreds of billions of pair comparisons.

M2 – API Rate Limiter

The basic operation is Audit(Request[j]). The outer variable $i$doubles during each iteration, and the inner loop executes $i$times.

Although the outer loop executes logarithmically many times, the inner loop does not execute $n$times in every iteration. Instead, its workload doubles each time. Therefore, the total work forms a geometric series and is linear in n.

M3 – Product Search

Version S1: Sequential search compares the search key with products one by one. If the product is near the beginning, fewer comparisons are required. If it is last or absent, up to $n$comparisons are required.

Version S2: Binary search compares the key with the middle element and discards half the remaining entries after each unsuccessful comparison. Consequently, the search space decreases exponentially, giving logarithmic time complexity.

M4 – Recommendation Engine

The current engine creates four recursive subproblems, each of size $n/2$, and performs $n$additional operations to merge their results. The proposed design removes replication and creates only two subproblems of size $n/2$. This reduces the number of recursive subproblems and improves scalability.

M5 – Sales Analytics Engine

The engine follows the recurrence $T(n) = 2T(n/2) + n \log n$. It creates two half-sized subproblems and performs $n \log n$additional work. Because this extra work is greater than

linear, the total running time is $\Theta(n\log^2 n)$. The separate report generator has a quadratic cost, as proved in Part (f).

(b) ANALYSE THE ITERATIVE MODULES

1. M1 – Exact Number of Comparisons

The duplicate detector compares every product with all products appearing after it.

For a catalogue containing $n$products:

- Product 0 is compared with $n-1$products.
- Product 1 is compared with $n-2$products.
- Product 2 is compared with $n-3$products.
- The final product has no later product to compare with.

Therefore, the total number of comparisons is

$$C(n) = (n-1) + (n-2) + \cdots + 2 + 1$$

Using the arithmetic-series formula,

$$1 + 2 + \cdots + (n-1) = \frac{(n-1)n}{2}$$

Hence,

$$\boxed{C(n) = \frac{n(n-1)}{2}}$$

Expanding,

$$C(n) = \frac{n^2 - n}{2}$$

$$C(n) = \frac{1}{2}n^2 - \frac{1}{2}n$$

The dominant term is $n^2/2$. Asymptotic analysis ignores constant factors and lower-order terms.

Therefore,

$$\boxed{T(n) = \Theta(n^2)}$$

Space Complexity

The algorithm uses loop variables and compares products without creating a separate collection of all pairs. Assuming MarkDuplicate uses constant auxiliary space,

$$\boxed{S(n) = \Theta(1)}$$

This is auxiliary space; the existing catalogue itself occupies $\Theta(n)$storage.

Example

For $n = 5$,

$$C(5) = \frac{5(5-1)}{2}$$

$$C(5) = \frac{5 \times 4}{2} = 10$$

Product selected	**Later products compared**	**Comparisons**
Product 1	Products 2–5	4
Product 2	Products 3–5	3
Product 3	Products 4–5	2
Product 4	Product 5	1
Product 5	None	0
Total		**10**

This demonstrates that the number of comparisons follows a triangular-number pattern.

2. M2 – Exact Number of Audit Operations

The algorithm starts with $i = 1$. After each inner loop, it doubles i.

The successive values of $i$are

$$1,2,4,8, \ldots$$

For each value of i, the inner loop executes exactly $i$times. Suppose the outer loop executes $k$times. Its values are

$$1,2,4, \ldots, 2^{k-1}$$

The loop continues while $i < n$, so $2^k$is the first power of two that is at least n. For $n > 1$,

$$k = \lceil \log_2 n \rceil$$

The total number of audit operations is

$$A(n) = 1 + 2 + 4 + \cdots + 2^{k-1}$$

Using the geometric-series formula,

$$1 + 2 + 4 + \cdots + 2^{k-1} = 2^k - 1$$

Therefore,

$$\boxed{A(n) = 2^{\lceil \log_2 n \rceil} - 1,\ n > 1}$$

For $n = 1$, the loop executes zero audit operations.

Is the Team Member Correct?

The team member claims that the complexity is $\Theta(n \log n)$, reasoning that the outer loop executes approximately $\log n$times and the inner loop executes up to $n$times. This reasoning is incorrect because the inner-loop lengths are not all n. They are 1,2,4,8, ...

For $n$greater than 1,

$$n \leq 2^{\lceil \log_2 n \rceil} < 2n$$

Subtracting 1 gives

$$n - 1 \leq A(n) < 2n - 1$$

Thus, the audit count is bounded above and below by positive constant multiples of $n$for sufficiently large n.

Hence,

$$\boxed{A(n) = \Theta(n)}$$

Conclusion: The team's $\Theta(n \log n)$claim is incorrect. The tight bound is $\boxed{\Theta(n)}$.

Verification for $n = 1,024$

Since

$$1024 = 2^{10}$$

the outer-loop values are

$$1,2,4,8,16,32,64,128,256,512$$

The number of outer-loop iterations is 10. The total number of audit operations is

$$A(1024) = 1 + 2 + 4 + \cdots + 512$$
$$A(1024) = 2^{10} - 1$$
$$\boxed{A(1024) = 1023}$$

The loop stops before $i = 1024$, because its condition is $i < n$.

Space Complexity

The algorithm uses only the variables $i$and j, so its auxiliary space complexity is

$$\boxed{S(n) = \Theta(1)}$$

(c) SOLVE A RECURRENCE BY SUBSTITUTION

1. Binary Search Recurrence

For search version S2, the given recurrence is

$$T(n) = T(n/2) + 1$$

with

$$T(1) = 1$$

Here, $T(n)$represents the number of comparisons needed to search a sorted index of size $n$under the stated recurrence. Each comparison eliminates half the remaining search space, and one comparison is added at every recursive level.

2. Backward Substitution

Starting with the original recurrence,

$$T(n) = T(n/2) + 1$$

Substitute the recurrence for $T(n/2)$:

$$T(n/2) = T(n/4) + 1$$

Therefore,

$$T(n) = T(n/4) + 1 + 1$$
$$T(n) = T(n/4) + 2$$

Substitute again:

$$T(n/4) = T(n/8) + 1$$

Thus,

$$T(n) = T(n/8) + 3$$

Continuing the same process,

$$T(n) = T(n/2^k) + k$$

where $k$is the number of substitution steps.

The recursion stops when the subproblem reaches size 1:

$$\frac{n}{2^k} = 1$$

Multiplying both sides by 2^k,

$$n = 2^k$$

Taking logarithms to base 2,

$$\log_2 n = k$$

Substituting this value into the recurrence,

$$T(n) = T(1) + \log_2 n$$

Since $T(1) = 1$,

$$\boxed{T(n) = \log_2 n + 1}$$

This exact closed form applies when n is a power of two under the given recurrence.

For arbitrary positive integer input sizes, the usual binary-search comparison count is logarithmic, with the exact count depending on the tree shape and the convention used for counting the final comparison.

Therefore,

$$\boxed{T(n) = \Theta(\log n)}$$

3. Iterative and Recursive Space Complexity

Iterative Binary Search

An iterative implementation repeatedly updates the lower and upper bounds within a loop. It does not create recursive function calls.

Its time complexity is

$$\boxed{\Theta(\log n)}$$

Its auxiliary space complexity is

$$\boxed{\Theta(1)}$$

Recursive Binary Search

A recursive implementation calls itself on half the remaining index. Each call occupies a stack frame until the search finishes. The number of recursive levels is logarithmic:

$$\text{Stack depth} \approx \log_2 n$$

Therefore,

$$\boxed{\text{Time} = \Theta(\log n)}$$

$$\boxed{\text{Auxiliary space} = \Theta(\log n)}$$

Practical Example

For a sorted index containing 1,024 products,

$$\log_2 1024 = 10$$

The given recurrence gives

$$T(1024) = 10 + 1 = 11$$

The distinction is important: the recurrence explicitly specifies $T(1) = 1$, so it gives 11 under its stated counting convention.

(d) APPLY THE MASTER THEOREM

The Master Theorem helps determine the asymptotic running time of divide-and-conquer algorithms of the form

$$T(n) = aT(n/b) + f(n)$$

where:

- $a$is the number of recursive subproblems.
- $n/b$is the size of each subproblem.
- $f(n)$is the non-recursive work.
- $n^{\log_b a}$is the critical comparison function.

1. M4 – Current Recommendation Engine

The recurrence is

$$T(n) = 4T(n/2) + n$$

Comparing it with the Master Theorem form gives

$$a = 4,\ b = 2,\ f(n) = n$$

Calculate the critical function:

$$n^{\log_b a} = n^{\log_2 4}$$

Since

$$\log_2 4 = 2$$

we obtain

$$n^{\log_2 4} = n^2$$

Compare $f(n)$with the critical function:

$$f(n) = n = O(n^{2-1})$$

Thus, $f(n)$is polynomially smaller than n^2.

Master Theorem Case 1 applies.

Therefore,

$$\boxed{T(n) = \Theta(n^2)}$$

The current recommendation engine has quadratic running time.

2. M4 – Proposed Recommendation Engine

The redesigned recurrence is

$$T(n) = 2T(n/2) + n$$

The parameters are

$$a = 2,\ b = 2,\ f(n) = n$$

Calculate the critical function:

$$n^{\log_b a} = n^{\log_2 2}$$

Since

$$\log_2 2 = 1$$

we obtain

$$n^{\log_2 a} = n$$

Therefore,

$$f(n) = \Theta\left(n^{\log_b a}\right)$$

Master Theorem Case 2 applies.

Hence,

$$\boxed{T(n) = \Theta(n \log n)}$$

The redesign improves the asymptotic running time from quadratic to linearithmic.

3. What Happens When the Data Doubles?

For the current design,

$$T(n) = \Theta(n^2)$$

Doubling the input gives

$$\frac{T(2n)}{T(n)} \approx \frac{(2n)^2}{n^2}$$

$$\boxed{\frac{T(2n)}{T(n)} \approx 4}$$

Thus, doubling the data approximately quadruples the dominant work. For the proposed design,

$$T(n) = \Theta(n\log_2 n)$$

The ratio becomes

$$\frac{T(2n)}{T(n)} \approx \frac{2n\log_2(2n)}{n\log_2 n}$$

Using

$$\log_2(2n) = 1 + \log_2 n$$

we obtain

$$\frac{T(2n)}{T(n)} \approx 2\left(1 + \frac{1}{\log_2 n}\right)$$

For large n, the ratio approaches 2. Therefore, the proposed design scales much better.

4. M5 – Sales Analytics Engine

The recurrence is

$$T(n) = 2T(n/2) + n\log n$$

The parameters are

$$a = 2,\ b = 2,\ f(n) = n\log n$$

The critical function is

$$n^{\log_b a} = n^{\log_2 2} = n$$

Compare the two functions:

$$f(n) = n\log n$$
$$n^{\log_b a} = n$$

The additional work is larger than the critical function by a logarithmic factor. Consequently, neither the standard Case 1 nor Case 2 condition applies directly.

The standard three-case version of the Master Theorem does not cover this exact logarithmic extension. The extended Master Theorem or a recursion tree can be used.

Recursion-Tree Derivation

At the first level, the non-recursive work is

$$n\log_2 n$$

At the second level, there are two subproblems of size $n/2$:

$$2\left(\frac{n}{2}\log_2\frac{n}{2}\right)$$

Since

$$\log_2\frac{n}{2} = \log_2 n - 1$$

the work is

$$n(\log_2 n - 1)$$

At the third level, there are four subproblems of size $n/4$:

$$4\left(\frac{n}{4}\log_2\frac{n}{4}\right)$$
$$= n(\log_2 n - 2)$$

Therefore, the work at level $i$is

$$W_i = n(\log_2 n - i)$$

The recursion has approximately $\log_2 n$levels.

Let $L = \log_2 n$. Summing the internal-level work gives

$$T(n) \propto n\sum_{i=0}^{L-1}(L - i)$$
$$T(n) \propto n(1 + 2 + \cdots + L)$$

Using the arithmetic-series formula,

$$T(n) \propto n\frac{L(L+1)}{2}$$

Since $L = \log_2 n$,

$$T(n) = \Theta(n(\log n)^2)$$

Therefore,

$$\boxed{T(n) = \Theta(n\log^2 n)}$$

The sales analytics engine has a higher asymptotic cost than a conventional merge-sort recurrence because each level performs $n \log n$work instead of linear work.

(e) ANALYSE BEST, AVERAGE AND WORST CASES

1. Search Version S1 – Sequential Search

Sequential search examines products one by one until it finds the required product or reaches the end.

Best Case

The product is the first item.

$$\boxed{C_{\text{best}}(n) = 1}$$

Time complexity:

$$\boxed{\Theta(1)}$$

Average Case When the Product Exists

Assume the target is equally likely to occur at any of the $n$positions.

The comparison counts are

$$1,2,3,\dots,n$$

The average number of comparisons is

$$C_{\text{avg}}(n) = \frac{1+2+\cdots+n}{n}$$

Using the arithmetic-series formula,

$$C_{\text{avg}}(n) = \frac{n(n+1)}{2n}$$

Therefore,

$$\boxed{C_{\text{avg}}(n) = \frac{n+1}{2}}$$

The average time complexity is $\Theta(n)$.

Worst Case

The product is last or does not exist.

$$\boxed{C_{\text{worst}}(n) = n}$$

Time complexity:

$$\boxed{\Theta(n)}$$

2. Average Comparisons When the Product Exists With Probability p

Let:

- $p$be the probability that the product exists.
- $1 - p$be the probability that it does not exist.

When the product exists, assume all $n$positions are equally likely. When it does not exist, sequential search checks all $n$products.

The conditional average when present is

$$E[C \mid \text{present}] = \frac{n+1}{2}$$

The comparison count when absent is

$$E[C \mid \text{absent}] = n$$

Using the law of total expectation,

$$E[C] = pE[C \mid \text{present}] + (1-p)E[C \mid \text{absent}]$$

Substituting,

$$E[C] = p\left(\frac{n+1}{2}\right) + (1-p)n$$

Expanding,

$$E[C] = \frac{p(n+1)}{2} + n - pn$$

Combining terms,

$$\boxed{E[C] = n - \frac{p(n-1)}{2}}$$

Evaluation for $n = 1,000,000$ and $p = 0.5$

Substitute the given values:

$$E[C] = 1{,}000{,}000 - \frac{0.5(1{,}000{,}000 - 1)}{2}$$

$$E[C] = 1{,}000{,}000 - \frac{0.5(999{,}999)}{2}$$

$$E[C] = 1{,}000{,}000 - 249{,}999.75$$

Thus,

$$\boxed{E[C] = 750{,}000.25}$$

The expected number of comparisons is approximately 750,000.

This is reasonable because half the searches find the product after an average of approximately 500,000 comparisons, while the other half perform 1,000,000 comparisons.

3. Search Version S2 – Binary Search

Binary search operates on a sorted index and discards half the remaining entries after each comparison.

Best Case

The target is at the middle of the index.

$$\boxed{C_{\text{best}}(n) = 1}$$

Average Case

For a balanced binary-search tree, the average search requires logarithmically many comparisons, assuming a suitable distribution of target positions.

$$\boxed{C_{\text{avg}}(n) = \Theta(\log n)}$$

The exact average depends on the target distribution and the tree structure.

Worst Case

The search repeatedly follows one half until the target is found or the search space becomes empty.

$$\boxed{C_{\text{worst}}(n) = \Theta(\log n)}$$

For the recurrence given in Part (c), the specified comparison count is $\log_2 n + 1$when $n$is a power of two.

4. Comparison of S1 and S2

Case	Sequential Search S1	Binary Search S2
Best	1 comparison	1 comparison
Average	$\Theta(n)$	$\Theta(\log n)$
Worst	$n$comparisons	$\Theta(\log n)$
Requires sorted data	No	Yes
Suitable for a million-item catalogue	Slow for repeated searches	Much more scalable

Binary search is the better option for frequent product searches when the sorted index can be maintained efficiently.

(f) PROVE AN ASYMPTOTIC BOUND

The report generator performs

$$f(n) = 3n^2 + 10n\log_2 n + 50$$

We must prove that

$$\boxed{f(n) = \Theta(n^2)}$$

1. Definition of Big-O

A function $f(n)$is $O\big(g(n)\big)$if there exist positive constants $c$and $n_0$such that

$$0 \leq f(n) \leq cg(n)$$

for every $n \geq n_0$.

2. Definition of Big-Omega

A function $f(n)$is $\Omega\big(g(n)\big)$if there exist positive constants $c$and $n_0$such that

$$0 \leq cg(n) \leq f(n)$$

for every $n \geq n_0$.

3. Definition of Big-Theta

A function $f(n)$is $\Theta\big(g(n)\big)$if positive constants c_1, c_2, n_0exist such that

$$0 \leq c_1 g(n) \leq f(n) \leq c_2 g(n)$$

for every $n \geq n_0$.

4. Proving the Upper Bound

For $n \geq 1$,

$$\log_2 n \leq n$$

Therefore,

$$n\log_2 n \leq n^2$$

Also, for $n \geq 1$,

$$50 \leq 50n^2$$

Starting with

$$f(n) = 3n^2 + 10n\log_2 n + 50$$

apply these inequalities:

$$f(n) \leq 3n^2 + 10n^2 + 50n^2$$
$$f(n) \leq 63n^2$$

Choose

$$\boxed{c_2 = 63}$$

and $n_0 = 1$. Thus,

$$f(n) \leq c_2 n^2$$

for every $n \geq 1$.

Hence,

$$\boxed{f(n) = O(n^2)}$$

5. Proving the Lower Bound

Since all terms are non-negative for $n \geq 1$,

$$f(n) = 3n^2 + 10n\log_2 n + 50$$

Therefore,

$$f(n) \geq 3n^2$$

Choose

$$\boxed{c_1 = 3}$$

and $n_0 = 1$. Then

$$3n^2 \leq f(n)$$

for every $n \geq 1$.

Hence,

$$\boxed{f(n) = \Omega(n^2)}$$

6. Final Big-Theta Proof

Combining both inequalities,

$$3n^2 \leq f(n) \leq 63n^2$$

for every $n \geq 1$.

Thus, valid constants are

$$\boxed{c_1 = 3,\ c_2 = 63,\ n_0 = 1}$$

Both the upper and lower bounds are satisfied. Therefore,

$$\boxed{f(n) = \Theta(n^2)}$$

7. Proving That $f(n)$Is Not $O(n)$

Suppose $f(n)$were $O(n)$. Then there would exist a positive constant $c$and an integer $n_0$such that

$$f(n) \leq cn$$

for every $n \geq n_0$.

However,

$$f(n) = 3n^2 + 10n\log_2 n + 50 \geq 3n^2$$

Consequently,

$$3n^2 \leq cn$$

Dividing both sides by $n > 0$,

$$3n \leq c$$

This would require a fixed constant $c$to be at least $3n$for every sufficiently large n. That is impossible because $3n$increases without bound.

Therefore,

$$\boxed{f(n) \notin O(n)}$$

Interpretation: The report generator's work grows quadratically, not linearly. Increasing the number of records tenfold can increase its dominant operation count by approximately one hundred times.

(g) SCALABILITY DECISION

1. Assumptions for Runtime Estimation

The server executes

$$10^8$$

basic operations per second.

The estimated execution time is

$$\boxed{\text{Time in seconds} = \frac{\text{Number of operations}}{10^8}}$$

The nightly processing limit is 10 minutes:

$$10 \times 60 = 600 \text{ seconds}$$

Thus, the approximate operation budget is

$$600 \times 10^8 = 6 \times 10^{10}$$

or

$$\boxed{60{,}000{,}000{,}000 \text{ basic operations}}$$

These estimates assume that the stated operation counts represent comparable units of work and exclude disk I/O, network delays, memory-allocation costs, and other overhead.

2. M1 – Duplicate-Listing Detector

For $n = 1{,}000{,}000$,

$$C(n) = \frac{n(n-1)}{2}$$

Substituting,

$$C(1{,}000{,}000) = \frac{1{,}000{,}000(999{,}999)}{2}$$

$$\boxed{C(1{,}000{,}000) = 499{,}999{,}500{,}000}$$

The estimated time is

$$t = \frac{499{,}999{,}500{,}000}{100{,}000{,}000}$$

$$\boxed{t = 4{,}999.995 \text{ seconds}}$$

Converting into minutes,

$$t = \frac{4{,}999.995}{60}$$

$$\boxed{t \approx 83.33 \text{ minutes}}$$

Decision: M1 fails the 10-minute requirement.

Recommendation: Replace the all-pairs comparison with a hash-based index keyed by seller identity and a normalized product title. This can reduce expected processing time to $\Theta(n)$, assuming effective hashing and manageable collision rates.

3. M2 – API Rate Limiter

For $n = 1{,}000{,}000$,

$$A(n) = 2^{\lceil \log_2 n \rceil} - 1$$

Since

$$2^{19} = 524{,}288$$

and

$$2^{20} = 1{,}048{,}576$$

we have

$$\lceil \log_2, 1, 000000 \rceil = 20$$

Therefore,

$$A(n) = 2^{20} - 1$$

$$\boxed{A(n) = 1{,}048{,}575}$$

The estimated time is

$$t = \frac{1{,}048{,}575}{10^8}$$

$$\boxed{t = 0.01048575 \text{ seconds}}$$

Decision: M2 meets the 10-minute requirement comfortably.

Recommendation: No redesign is required for asymptotic efficiency. The incorrect $\Theta(n \log n)$claim should be corrected to $\Theta(n)$. Operational monitoring is still advisable.

4. M3 – Binary Search S2

The recurrence specifies

$$T(n) = \log_2 n + 1$$

for powers of two. Since one million is not a power of two, the actual count depends on the implementation. A conservative logarithmic estimate is approximately 21 comparisons.

For a standard binary search on one million entries,

$$\lceil \log_2(1,000,000 + 1) \rceil = 20$$

is sufficient as a bound on the number of comparisons needed to find an existing key in the usual implementation, while some unsuccessful-search conventions can require a further comparison. For a simple operational estimate, approximately 20–21 comparisons is appropriate.

Using 21 comparisons,

$$t = \frac{21}{10^8}$$

$$\boxed{t = 2.1 \times 10^{-7} \text{ seconds}}$$

This is the idealized operation-based estimate, not a guaranteed real-world latency.

Decision: S2 meets the nightly processing requirement easily.

Recommendation: Retain binary search and maintain the sorted index. Consider the cost of index updates when products are inserted, removed, or modified.

5. M4 – Current Recommendation Engine

The recurrence is

$$T(n) = 4T(n/2) + n$$

For powers of two, its expansion gives

$$T(n) = n^2 + n\log_2 n$$

assuming $T(1) = 1$.

For the purpose of an order-of-growth estimate at $n = 1{,}000{,}000$,

$$n^2 = 10^{12}$$

and

$$n\log_2 n \approx 19{,}931{,}568.6$$

Therefore, the dominant term is approximately

$$\boxed{10^{12} \text{ operations}}$$

The idealized runtime is approximately

$$t \approx \frac{10^{12}}{10^8}$$

$$\boxed{t \approx 10{,}000 \text{ seconds}}$$

Converting to minutes,

$$t \approx 166.67 \text{ minutes}$$

Decision: The current design fails the nightly processing requirement.

Recommendation: Remove unnecessary replication, reduce the number of recursive subproblems from four to two, and use the proposed $\Theta(n \log n)$design where the data semantics permit it.

6. M4 – Proposed Recommendation Engine

The redesigned recurrence is

$$T(n) = 2T(n/2) + n$$

For powers of two, with $T(1) = 1$, the exact solution is

$$T(n) = n\log_2 n + n$$

At $n = 1{,}000{,}000$,

$$T(n) \approx 1{,}000{,}000(19.931569) + 1{,}000{,}000$$

$$\boxed{T(n) \approx 20{,}931{,}569 \text{ operations}}$$

The idealized runtime is

$$t = \frac{20{,}931{,}569}{10^8}$$

$$\boxed{t \approx 0.2093 \text{ seconds}}$$

Decision: The proposed design meets the processing requirement under the stated operation-count model.

Recommendation: Adopt the redesigned recurrence if removing replication does not compromise correctness, fault tolerance, or recommendation quality. Validate these requirements through testing before deployment.

7. M5 – Sales Analytics Engine

The recurrence is

$$T(n) = 2T(n/2) + n\log_2 n$$

From the recursion-tree analysis, the complexity is

$$\boxed{T(n) = \Theta(n\log^2 n)}$$

At $n = 1{,}000{,}000$,

$$\log_2 n \approx 19.931569$$

Therefore, the order-of-growth expression is

$$n(\log_2 n)^2$$
$$\approx 1{,}000{,}000(19.931569)^2$$
$$\boxed{\approx 397{,}267{,}424}$$

This is an asymptotic estimate, not an exact operation count. Its idealized time equivalent is

$$t \approx \frac{397{,}267{,}424}{10^8}$$

$$\boxed{t \approx 3.97 \text{ seconds}}$$

The actual runtime cannot be established exactly without the recurrence's base case, implementation details, and the constant factors hidden by the Θ-notation.

Decision: The recurrence's growth estimate is below the 10-minute budget under the stated idealized model, but actual performance must be measured before confirming compliance.

Recommendation: Profile the engine, optimize repeated sorting and merging, and investigate whether the required $n \log n$work at every level can be reduced. Avoid claiming an exact runtime from asymptotic complexity alone.

8. Overall Scalability Comparison

Module/design	**Estimated operations at $n = 1{,}000{,}000$**	**Idealized time**	**10-minute decision**
M1 – Duplicate detector	499,999,500,000	83.33 minutes	Fails
M2 – API rate limiter	1,048,575	0.0105 seconds	Meets
M3 S2 – Binary search	About 20–21 comparisons per search	About 0.0000002seconds per search	Meets
M4 – Current design	Approximately 10^{12}dominant operations	About 166.67 minutes	Fails
M4 – Proposed design	Approximately 20,931,569	About 0.2093 seconds	Meets
M5 – Sales analytics	Approximately 397 million as a growth estimate	About 3.97 seconds as a growth estimate	Requires profiling

The M3 estimate is per search, whereas the other estimates refer to their respective module workloads. These quantities should not be interpreted as identical workloads.

9. Final Recommendations

The most urgent redesigns are M1 and the current M4 recommendation engine.

M1 should replace all-pairs comparison with a hash-based duplicate-detection approach. This avoids quadratic growth and substantially reduces the expected number of comparisons.

M4 should remove unnecessary replicated subproblems where possible. Its proposed recurrence reduces the growth from $\Theta(n^2)$to $\Theta(n \log n)$, providing a major scalability improvement.

M2 already has linear growth and requires no asymptotic redesign. M3's binary-search implementation is efficient for a maintained sorted index. M5 should be profiled and optimized, but the supplied recurrence alone does not justify declaring that it fails the nightly processing limit.

Conclusion

The ShopSphere audit demonstrates that algorithm selection directly affects the scalability of an e-commerce platform. M1 and the current M4 design exhibit quadratic growth and exceed the idealized 10-minute processing budget at one million items. M2, M3 S2, and the proposed M4 design have much more favorable growth rates. M5 has $\Theta(n\log^2 n)$complexity and should be validated through realistic performance measurements.

The recommended strategy is to redesign the quadratic modules first, retain efficient logarithmic and linear algorithms, and use profiling to confirm the actual performance of the analytics engine.